

Arrays, Multidimensional Matrices and Memory Layout in C

> **Definition to Remember**

> An a
index a

1. Concept Overview & Fundamentals

Arrays, Multidimensional Matrices and Memory Layout in C forms a core cornerstone of computer systems programming, hardware-level software engineering, and competitive problem solving.

In the C execution model:

- Co

2. Technical Architecture & Memory Mechanics

Memory Layout & Storage Organization

```text

+-----

### Core Architecture Breakdown

| Component / Concept | Technical Specification | Operational Characteristics | Performance / Complexity |

| :--- | :

---

3. Core Syntax & Implementation Patterns

Essential Idioms & Header Declarations

```
```c
// Core standard implementation pattern
// Core logic execution
return status;
```
#include
int mai
printf("
}
```

---

4. Real-World Applications & System Engineering

- **Operating System Kernels:** Linux, UNIX, and Windows NT kernels utilize C for drivers, process scheduling, and memory paging.

- **E**

---

5. Comprehensive Solved Problems & Code Walkthroughs

Problem 1: array max min

**Problem Description & Objective:**

**\*\*C Code Implementation:\*\***

```
#include <stdio.h>
```

```
int main() {
```

```
int arr[5], i;
```

```
printf("Enter 5 elements:\n");
```

```
for(i = 0; i < 5; i++) {
```

```
max = arr[0];
```

```
for(i = 1; i < 5; i++) {
```

```
if(arr[i] > max) {
```

```
if(arr[i] < min) {
```

```
printf("Maximum element = %d\n", max);
```

```
return 0;
```

```

```

## **Problem 2: array occurrences**

**\*\*Problem Description & Objective:\*\***

**\*\*C Code Implementation:\*\***

```
#include <stdio.h>
```

```
int main() {
```

```
int arr[10], i, num, count = 0;
```

```
printf("Enter 10 elements:\n");
```

```
for(i = 0; i < 10; i++) {
```

```
printf("Enter element to search: ");
```

```
for(i = 0; i < 10; i++) {
```

```
if(arr[i] == num) {
```

```
printf("Number of occurrences = %d\n", count);
```

```
return 0;
```

```

```

### **Problem 3: array palindrome**

**\*\*Problem Description & Objective:\*\***

**\*\*C Code Implementation:\*\***

```
#include <stdio.h>
```

```
int main() {
```

```
/*
```

```
int arr[5], i;
```

```
int isPa
```

```
printf("Enter 5 elements:\n");
```

```
for(i = 0; i < 5; i++) {
```

```
scanf("
```

```
for(i = 0; i < 5 / 2; i++) {
```

```
if(arr[i] != arr[5 - i - 1]) {
```

```
isPalin
```

```
if(isPalindrome == 1) {
```

```
printf("
```

```
return 0;
```

```
}
```

```

```

#### **Problem 4: array sorted check**

**\*\*Problem Description & Objective:\*\***

Implem

**\*\*C Code Implementation:\*\***

```
```c
```

```
#include <stdio.h>
```

```
int main() {
```

```
/*
```

```
int arr[5], i;
```

```
printf("Enter 5 elements:\n");
```

```
for(i = 0; i < 5; i++) {
```

```
for(i = 0; i < 4; i++) {
```

```
if(arr[i] > arr[i + 1]) {
```

```
if(isSorted == 1) {
```

```
return 0;
```

```
---
```

Problem 5: array sum average

****Problem Description & Objective:****

****C Code Implementation:****

```
#include <stdio.h>
```

```
int main() {
```

```
int arr[5], i;
```

```
int sum
```

```
printf("Enter 5 elements:\n");
```

```
for(i = 0; i < 5; i++) {
```

```
scanf("%d", &arr[i]);
```

```
average = sum / 5.0;
```

```
printf("Sum = %d\n", sum);
```

```
printf("Average = %.2f\n", average);
```

```
return 0;
```

```
}
```

```
---
```

Problem 6: copy char array pointer

****Problem Description & Objective:****

Implement a function to copy a string from one array to another.

****C Code Implementation:****

```
```c
```

```
#include <stdio.h>
```

```
// Function Definition
```

```
void copy(char *source, char *destination)
```

```
source++;
```

```
destination++;
```

```
*destination = '\0';
```

```
}
```

```
int main() {
```

```
printf("Enter a string: ");
```

```
copyString(str1, str2);
```

```
printf("Copied string = %s\n", str2);
```

```
return 0;
```

char str

scanf(

}

---

### Problem 7: delete array element

**\*\*Problem Description & Objective:\*\***

Implem

**\*\*C Code Implementation:\*\***

```c

```
#include <stdio.h>
```

```
int main() {
```

/*

```
printf("Enter number of elements (1-10): ");
```

if (scan

```
if (n < 1 || n > 10) {
```

printf("

```
printf("Enter array elements:\n");
```

```
for(i = 0; i < n; i++) {
```

```
printf("Enter element to delete: ");
```

```
for(i = 0; i < n; i++) {
```

```
if(arr[i] != element) {
```

```
printf("New array:\n");
```

```
for(i = 0; i < j; i++) {
```

```
return 0;
```

```
---
```

Problem 8: diagonal sum

****Problem Description & Objective:****

****C Code Implementation:****

```
#include <stdio.h>
```

```
int main() {
```

```
int arr[3][3];
```

```
printf("Enter 3x3 matrix elements:\n");
```

```
for(i = 0; i < 3; i++) {
```

```
for(j =
```

```
// Primary diagonal sum
```

```
for(i =
```

```
// Secondary diagonal sum
```

```
for(i =
```

```
printf("Primary Diagonal Sum = %d\n", primarySum);
```

```
printf("
```

```
return 0;
```

```
}
```

```
---
```

Problem 9: merge sorted arrays

****Problem Description & Objective:****

Implemen

****C Code Implementation:****

```
```c
```

```
#include <stdio.h>
```

```
int main() {
```

```
/*
```

```
int arr1[5], arr2[5], merged[10];
```

```
int i, j,
```

```
printf("Enter 5 sorted elements for first array:\n");
```

```
for(i = 0; i < 5; i++) {
```

```
scanf("%d", &arr2[j]);
```

```
printf("Enter 5 sorted elements for second array:\n");
```

```
for(i = 0; i < 5; i++) {
```

```
scanf("%d", &arr2[j]);
```

```
i = 0;
```

```
j = 0;
```

```
while(i < 5 && j < 5) {
```

```
if(arr1[i] < arr2[j]) {
```

```
merge[k] = arr1[i];
```

```
k++;
```

```
}
```

```
while(i < 5) {
```

```
merge[k] = arr1[i];
```

```
while(j < 5) {
```

```
merge[k] = arr2[j];
```

```
printf("Merged array:\n");
```

```
for(i = 0; i < 10; i++) {
```

```
printf("%d ", merge[i]);
```

```
return 0;
```

```
}
```

```

```

### Problem 10: reverse array

**\*\*Problem Description & Objective:\*\***

Implement

**\*\*C Code Implementation:\*\***

```
```c
```

```
#include <stdio.h>
```

```
int main() {
```

```
/*
```

```
int arr[5], i;
```

```
printf("Enter 5 elements:\n");
```

```
for(i = 0; i < 5; i++) {
```

```
scanf("%d", &arr[i]);
```

```
printf("Reversed array:\n");
```

```
for(i = 4; i >= 0; i--) {
```

```
printf("%d ", arr[i]);
```

```
return 0;
```

```
}
```

```
---
```

Problem 11: search 2d array

****Problem Description & Objective:****

Implement a function to search an element in a 2D array.

****C Code Implementation:****

```
```c
```

```
#include <stdio.h>
```

```
int main() {
```

```
/*
```

```
int arr[3][3];
```

```
int i, j,
```

```
printf("Enter 3x3 matrix elements:\n");
```

```
for(i = 0; i < 3; i++) {
```

```
for(j =
```

```
printf("Enter element to search: ");
```

```
scanf(")
```

```
for(i = 0; i < 3; i++) {
```

```
for(j = 0; j < 3; j++) {
```

```
if(arr[i][j] == num) {
```

```
printf(")
```

```
if(found == 0) {
```

```
printf(")
```

```
return 0;
```

```
}
```

```

```

### Problem 12: sum average 2d array

**\*\*Problem Description & Objective:\*\***

Implemen

**\*\*C Code Implementation:\*\***

```
```c
```

```
#include <stdio.h>
```

```
int main() {
```

```
/*
```

```
int arr[3][3];
```

```
int i, j,
```

```
printf("Enter 3x3 matrix elements:\n");
```

```
for(i = 0; i < 3; i++) {
```

```
for(j =
```

```
sum = sum + arr[i][j];
```

```
}
```

```
average = sum / 9.0;
```

```
printf("Sum = %d\n", sum);
```

```
printf("
```

```
return 0;
```

```
}
```

```
---
```

6. Exam Scoring Points & Memory Revision

```
> **Must-Write Points**
```

```
> 1. C
```

```
> **Quick Recall**
```

```
> `Bas
```